

#81 FaaSpt: Fault-Tolerant Serverless Workflow Execution on Spot Instances

**Main**[Edit](#)[Your submissions](#)

Email notification

Select to receive email on updates to reviews and comments.

▼ PC conflicts

Abhishek Chandra
Kwangsung Oh

Submitted



Submission (1.3MB) · Jun 5, 2026, 5:07:55 AM AoE · 43837f78

► Abstract

Serverless computing is increasingly

► Authors (anonymous)

B. Patel, S. Harsha, K. Oh, I. Kim [\[details\]](#)

► Topics and options

[\[more\]](#)

[Review #81A](#)

[Review #81B](#)

[Review #81C](#)

[Review #81D](#)

[Review #81E](#)

You are an **author** of this submission.

[Edit submission](#)[Add Rebuttal response](#)



Review #81A

Paper summary

The paper introduces FaaSspot, a runtime system that employs several strategies to reduce the cost of preemptions in cloud-based scientific workflows running on spot instances. By applying three approaches, selective checkpointing, smart scaling, and different retry priority levels, it increases workflow execution success and lowers economic cost compared with existing systems. The authors evaluate FaaSspot through numerous experiments against four baseline systems across various metrics and present a trace-driven spot emulator that enables fair comparisons between the systems.

Strengths

- The work defines clear design principles and implements three complementary controllers: selective checkpointing, survival-aware scaling, and a lineage-based retry manager. The ablation study shows that the lineage-based retry manager is the dominant factor for reliability and performance.
- A spot-instance emulator is provided and used to evaluate the approach under preemption patterns derived from a real dataset.
- The experiments are exhaustive. They cover a wide variety of workflows, compare different serverless workflow execution systems across several metrics (success rate, latency, economic cost), and include an ablation study.

Weaknesses

- The approach requires some prior knowledge: the full workflow DAG must be known, task execution times must be available, and a history of the spot-instance service is needed.
- DAG-topology information alone may be insufficient to determine checkpoint locations. Task execution time can be an important feature. In the worst case, a workflow contains computationally heavy sequential tasks that the current checkpoint analysis does not cover, forcing re-execution in case of failure.

Comments for authors

The paper is pleasant to read; it presents a solid system with thorough experiments. However, it would benefit from a deeper discussion of whether task-execution time should be considered when selecting checkpoint locations, or at least an explanation for its omission (e.g., an experiment confirming that time is not so helpful). The remaining minor issues can be resolved by providing clearer explanations and defining the missing information.

- The authors treat task execution time as a secondary factor and focus exclusively on the topological impact of a preemption. Consequently, the paper does not analyze how execution time influences failure probability, even though the data in Figure 7 could support such an analysis. Checkpoint decisions should also depend on execution time; for example, a single sequential task that runs for 60 minutes deserves more aggressive protection than 100 parallel tasks, each running for only 10 seconds.
- The limitation of the static DAG is discussed in the paper. However, it is important to underline that static workflows are not so common. Also, the workflows described in the paper are static because some preprocessing analyzing the input dataset is made (in particular, 1000 genome and montage). Using another input dataset, the preprocessing must be re-executed.
- The HEFT algorithm used by FaaSspot (Equation 1) depends on the execution time of each task. Therefore, the system requires a reliable estimate of the execution time of all the tasks. If the actual runtime is unknown or highly variable, this assumption becomes a limitation. The offline DAG analysis may misrank tasks, classifying a low-impact node as critical (or vice-versa). Since task-time information is required for ranking, the same information could also improve checkpoint selection. Alternatively, checkpoint decisions that combine DAG topology and task time could be made at runtime after a task completes.
- The paragraph that introduces the "task ranker" presents only the HEFT rank formula; the scaling factor $k(v)$ formula is described but not defined. The paper should either present the complete equation for $k(v)$ or provide a short pseudo-code snippet.
- Table 1 lists the retry actions for the MEDIUM and CRITICAL tiers, but the surrounding text does not discuss them. Moreover, Algorithm 3 does not make the tier-specific retry actions explicit. The authors should clarify how the tiers are incorporated into Algorithm 3.
- In a pure pipeline, all tasks have the LOW tier because every task has $\text{minScale} = 1$. Under the current policy, LOW-tier tasks receive no checkpoints, which may leave the pipeline vulnerable to restarting from the beginning on every failure. The authors should discuss how the policy could be adapted for pipeline-style workflows.

- Explain how the scaling-factor ranges for each survival state were determined (e.g., empirically derived), and why the intervals of different survival states overlap rather than being strictly disjoint.
- In the success-rate experiment, every method employs a retry mechanism, so with unlimited time all of them would eventually terminate. Nevertheless, the reported success rates are below 100 %. This suggests that a retry limit was imposed. The authors should explicitly state the retry limit used in the experiments, or otherwise explain why the success rates are not 100 % despite the presence of a retry mechanism.
- A suggestion regarding Equation 3 is to consider also the total number of tasks in the workflow together with (n_c) (the number of completed tasks in the workflow) (e.g., normalizing completed tasks by the workflow size). It could make the metric more informative. Such a normalization would give higher priority to a short workflow that is near completion (e.g., 8 of 10 tasks) than to a long workflow that has completed more tasks (e.g., 9 of 50 tasks). If a similar effect is already achieved through the (w_c) term, the authors should clarify the relationship.

Review #81B

Paper summary

This paper proposes FaaSspot, a workflow execution platform that runs workflows on pure spot instances, yet ensures a good level of completion rate and low cost by analyzing the workflow DAG, ranking the tasks, and then applying various FT techniques to them, including checkpointing, replication/scaling, and retries, based on their criticality score. The work includes a prototype that is evaluated against other SOTA using emulation.

Strengths

The writing is very clear and the contribution is well motivated and the solution is well explained.

Includes an implementation and a detailed evaluation over several metrics plus an ablation study.

Demonstrated improvements over SOTA over multiple workloads.

Weaknesses

The work includes several arbitrary decisions (e.g., the ranking mechanism and the spot survivability prediction mechanism) as well as multiple magic numbers (e.g., Table 1, Table 2, Table 3) with no sensitivity analysis.

Only emulation, but this is reasonable considering the work done.

Comments for authors

Caption of Figure 2: the text for the (b) sub-figure does not seem to match the numbers displayed here.

Section 6: throughput, it would be good to include the results for the on-demand instances, as this is your absolute baseline that you are competing with.

Review #81C

Paper summary

This paper presents FaaS_{Spot}, a serverless workflow execution system that runs microservice applications entirely on spot instances. The primary challenge addressed by FaaS_{Spot} is that spot instance preemptions can interrupt the execution of microservice tasks before they complete. To mitigate this problem, FaaS_{Spot} exploits the topology of workflow DAGs and ranks application tasks according to the impact of their failures. It further introduces three mechanisms: selective checkpointing, survival-aware scaling, and lineage-based retry. The authors evaluate FaaS_{Spot} using a trace-driven spot instance emulator. The experimental results show that FaaS_{Spot} achieves high task completion rates while significantly reducing execution costs.

Strengths

- + Well organized and easy to follow.
- + tackle an important problem: how to build cost-efficient microservice applications in cloud environments.
- + Quantitatively demonstrate the effectiveness of FaaS_{Spot} through a comprehensive experimental evaluation.

Weaknesses

- The target applications are unclear.
- Storage cost for checkpointing is unclear.

Comments for authors

Microservices have become one of the dominant architectures for cloud-based web applications. This paper investigates how to reduce the cost of deploying microservice applications in cloud environments. It adopts a radical approach by executing application tasks entirely on spot instances, making the work both interesting and practically relevant. The paper offers a meaningful academic contribution, is well organized, and is easy to follow.

What are the target or "killer" applications envisioned for FaaS_{Spot}? Since FaaS_{Spot} relies entirely on spot instances, temporary service interruptions due to instance preemptions appear to be unavoidable. What kinds of applications can tolerate such interruptions? In what deployment scenarios would occasional service disruptions be considered acceptable? Clarifying the target applications and usage scenarios would help readers better understand the motivation for FaaS_{Spot} and the practical significance of the proposed approach.

The storage requirements and associated costs also deserve further discussion. FaaS_{Spot} relies on persistent storage to maintain checkpoints, making persistent storage an essential component of the overall system. What are the requirements for this storage? In cloud environments, persistent storage may be provided as block storage, file storage, or object storage, each offering different performance characteristics and pricing models. Since storage performance directly affects checkpointing overhead, it would be helpful to clarify which type of storage is assumed and whether its cost is included in the evaluation. Such a discussion would provide a more complete picture of the overall cost-effectiveness of FaaS_{Spot}. Related to this point, because FaaS_{Spot} depends on persistent storage, the phrase "runs entirely on spot instances" may be slightly misleading. It may be worth revising this wording to more accurately describe the system architecture.

Review #81D

Strengths

Simple solution Good motivation

Weaknesses

Limited scientific innovation

Comments for authors

This is a well-done paper. The topic is interesting, and the solution is simple and seems to work. Really, my only concern is that the level of innovation is not a breakthrough. FaaSspot is a well-executed integration of working ideas, meaning offline analysis of the workflow and checkpointing. This is not a negative, which is why my feedback is weak accept.

I haven't seen a conversation about persistent memory. Would that help resume execution of spot instances much quicker?

Also, what are the limitations of the DAG analysis approach? Which workflows don't fit well with the proposed process?

I think the evaluation study is well done.

Review #81E

Paper summary

This paper presents FaaSspot, a serverless workflow execution system on spot instances. It addresses cascading failures due to spot preemptions by identifying the high-impact tasks offline and designing three online fault-tolerance mechanisms. Evaluation based on a trace-driven emulator and 8 DAG workflows shows that FaaSspot outperforms baselines on success rate, cost, and latency.

Strengths

- + Enhancing the resilience of serverless workflow execution on spot instances is an important problem.
- + FaaSspot presents a practical solution that improves the resilience of serverless workflow execution on spot instances.

- + The evaluation includes eight representative workflows and demonstrates the effectiveness of FaaSspot under different settings.

Weaknesses

- The evaluation is conducted using an emulator rather than a real-world deployment. Since FaaSspot relies on fault-tolerance techniques such as checkpointing, scaling, and retry, evaluating the system in a real environment is necessary to show its practicality and effectiveness.
- The key factors that differentiate cascading failures in serverless workflows on spot instances from other DAG-based services are spot instance preemption and serverless cold starts. However, the paper does not evaluate the accuracy of the KM model, and the impact of serverless cold starts (e.g., during scaling) is not discussed.
- Since FaaSspot relies on checkpointing for fault recovery, the paper should discuss how checkpoint data is protected against failures.

Comments for authors

Thank you for submitting your work to Middleware '26. Enhancing the fault-tolerance of serverless workflow execution on spot instances is an important problem. My main concern is the evaluation methodology. The experiments are conducted using an emulator rather than a real-world deployment. Since FaaSspot relies on techniques such as checkpointing, scaling, and retry, evaluating the system in a real environment would provide stronger evidence of its practicality and effectiveness. In addition, the paper does not evaluate the accuracy of the KM model, making it difficult to understand how prediction quality affects the overall system. The discussion of cascading failures could also be improved. Spot instance preemption and serverless cold starts are two key factors in this setting, but the impact of cold starts, such as those introduced during scaling, is not discussed. Finally, since FaaSspot relies on checkpointing for fault recovery, it would be helpful to discuss how checkpoint data is protected against failures. For example, what happens if a failure occurs while creating, storing, or recovering a checkpoint?